

Projet de Fin d'Études

En vue de l'obtention du Diplôme de Licence en Informatique

Option : Informatique

THÈME :

**Conception et Implémentation d'un Cadre Décisionnel
basé sur l'Intelligence Artificielle pour l'Optimisation
du Suivi Parcellaire Agricole au Tchad
(Irrigation, Fertilisation et Détection des Maladies)**

Rédigé et Présenté par :

TOURABI ISSAK HISSEIN

Sous la Direction de :

Dr. MOUAZ MIKAIL,
Enseignant Chercheur à ENASTIC

Jury :

..... (Président)

..... (Rapporteur)

Année Académique 2025–2026

Dédicace

À mes chers parents,

*pour leur amour inconditionnel,
leurs sacrifices et leur soutien indéfectible
tout au long de mon parcours académique.*

Remerciements

Au terme de ce travail, il m'est agréable d'exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Mes premiers remerciements vont à **Dr. MOUAZ MIKAIL**, mon encadreur, pour sa disponibilité permanente, ses précieux conseils scientifiques et sa rigueur méthodologique qui ont grandement orienté la conduite de ce travail. Sa patience et son expertise ont été une source d'inspiration tout au long de cette expérience.

Je remercie sincèrement les membres du jury qui ont accepté d'évaluer ce travail et d'apporter leurs critiques constructives pour son amélioration.

Ma gratitude va également à l'ensemble du corps enseignant et administratif de l'**École Nationale Supérieur des TIC (ENASTIC)** pour la qualité de la formation académique et professionnelle qu'ils m'ont dispensée durant mon cursus.

Je tiens à remercier chaleureusement mes camarades de promotion pour l'ambiance studieuse et fraternelle, ainsi que les échanges enrichissants qui ont jalonné notre formation.

Enfin, mes remerciements les plus profonds vont à ma famille, en particulier à mes parents et à mon oncle HASSAN HISSEIN DAOUD, pour leur soutien moral et matériel indéfectible, et pour tous les sacrifices consentis afin de permettre la réussite de mes études.

Que tous ceux dont le nom ne figure pas ici, mais qui ont contribué d'une manière ou d'une autre à l'élaboration de ce mémoire, trouvent ici l'expression de ma sincère reconnaissance.

Résumé

L'agriculture tchadienne, pilier de l'économie nationale, souffre d'un déficit d'outils d'aide à la décision adaptés aux réalités locales. Ce mémoire présente la conception et l'implémentation d'un cadre décisionnel basé sur l'Intelligence Artificielle pour l'optimisation du suivi parcellaire agricole au Tchad, dénommé **Agro-IA Tchad**.

Le système intègre quatre modules complémentaires. Le **module Irrigation** repose sur l'algorithme Random Forest, entraîné sur 16 435 observations climatiques réelles issues de la base NASA POWER pour cinq villes tchadiennes (2015–2023) ; il atteint une accuracy de 78,51 % et un AUC-ROC de 0,8782. Le **module Fertilisation** utilise XGBoost sur 1 599 observations et recommande l'engrais optimal avec une accuracy de 95,42 % (F1-score macro de 95,47 %). Le **module Détection des maladies** combine un réseau de neurones convolutif MobileNetV2 — entraîné par transfert d'apprentissage sur un dataset local de 287 images de cultures tchadiennes (accuracy de 52,27 %) — avec l'analyse d'indices de végétation NDVI issus du satellite Sentinel-2 ou estimés par caméra smartphone (indice VARI). Enfin, le **module Historique** assure la traçabilité de toutes les analyses grâce à une base de données SQLite avec export CSV.

L'application web, développée avec le framework Streamlit et intégrant l'API météorologique Open-Meteo en temps réel, est déployée sur Streamlit Cloud et accessible depuis tout navigateur.

Mots-clés : Intelligence Artificielle, Random Forest, XGBoost, CNN MobileNetV2, NDVI, Agriculture de précision, Tchad, Streamlit, SQLite.

Abstract

Chadian agriculture, a pillar of the national economy, suffers from a lack of decision-support tools adapted to local realities. This thesis presents the design and implementation of an Artificial Intelligence-based decision framework for optimizing agricultural plot monitoring in Chad, named **Agro-IA Tchad**.

The system integrates four complementary modules. The **Irrigation module** is based on the Random Forest algorithm, trained on 16,435 real climate observations from the NASA POWER database for five Chadian cities (2015–2023), achieving 78.51 % accuracy and an AUC-ROC of 0.8782. The **Fertilization module** uses XGBoost on 1,599 observations and recommends the optimal fertilizer with 95.42 % accuracy (macro F1-score of 95.47 %). The **Disease Detection module** combines a MobileNetV2 convolutional neural network — trained by transfer learning on a local dataset of 287 images of Chadian crops (52.27 % accuracy) — with NDVI vegetation indices from the Sentinel-2 satellite or estimated via smartphone camera (VARI index). Finally, the **History module** ensures traceability of all analyses through a SQLite database with CSV export.

The web application, developed with the Streamlit framework and integrating the Open-Meteo real-time weather API, is deployed on Streamlit Cloud and accessible from any browser.

Keywords : Artificial Intelligence, Random Forest, XGBoost, CNN MobileNetV2, NDVI, Precision Agriculture, Chad, Streamlit, SQLite.

Sommaire

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Sommaire	v
Liste des figures	vii
Liste des tableaux	viii
Liste des abréviations	ix
Introduction Générale	1
1 Travaux existants	4
2 Analyse, Conception et Modélisation UML	9
3 Implémentation et Tests	18
4 Résultats et Discussion	25
Conclusion Générale et Perspectives	29
Bibliographie	31
A Jeux de données utilisés	33
B Codes sources principaux	35
C Diagrammes UML complets	37
D Captures d'écran de l'application	38

Table des matières	41
---------------------------	-----------

Table des figures

2.1	Diagramme de contexte du système Agro-IA Tchad	11
2.2	Diagramme de cas d'utilisation global avec services externes	12
2.3	Diagramme de séquence — analyse d'une parcelle	13
2.4	Diagramme d'activités — génération d'une recommandation	14
2.5	Architecture générale du système Agro-IA Tchad	15
2.6	Architecture en couches du cadre décisionnel Agro-IA Tchad	15
D.1	Page d'accueil de l'application Agro-IA Tchad	38
D.2	Module Irrigation — analyse et recommandation	39
D.3	Module Fertilisation — recommandation d'engrais	39
D.4	Module Détection des maladies — diagnostic par photo	40
D.5	Module Historique — consultation et export	40

Liste des tableaux

1.1	Comparaison des solutions existantes avec Agro-IA Tchad	7
2.1	Besoins fonctionnels du système Agro-IA Tchad	10
2.2	Structure de la table irrigation	16
2.3	Structure de la table fertilisation	16
2.4	Structure de la table maladies	17
3.1	Environnements de développement utilisés	18
4.1	Performances du module Irrigation (Random Forest)	25
4.2	Performances du module Fertilisation (XGBoost)	26
4.3	Performances du CNN MobileNetV2 (dataset tchadien)	26
4.4	Résultats des tests fonctionnels du module Historique	27
4.5	Synthèse des performances des trois modules d'IA	27
A.1	Caractéristiques du dataset d'irrigation	33
A.2	Caractéristiques du dataset de fertilisation	33
A.3	Distribution du dataset local tchadien (287 images)	34

Liste des Abréviations

API	Application Programming Interface
AUC	Area Under the Curve
CNN	Convolutional Neural Network (Réseau de Neurones Convolutif)
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Cross-Validation (Validation Croisée)
DAP	Diammonium Phosphate
DL	Deep Learning (Apprentissage Profond)
ENASTIC	École Nationale Supérieure des Technologies de l'Information et de la Communication
ESA	European Space Agency
ETP	Évapotranspiration Potentielle
GPS	Global Positioning System
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
ITRAD	Institut Tchadien de Recherche Agronomique pour le Développement
KCl	Chlorure de Potassium
ML	Machine Learning (Apprentissage Automatique)
NASA	National Aeronautics and Space Administration
NDVI	Normalized Difference Vegetation Index
NIR	Near InfraRed (Proche Infrarouge)
NPK	Azote (N), Phosphore (P), Potassium (K)
PFE	Projet de Fin d'Études
PIB	Produit Intérieur Brut
RF	Random Forest (Forêt Aléatoire)
RGB	Red Green Blue (Rouge Vert Bleu)
ROC	Receiver Operating Characteristic
SGBD	Système de Gestion de Base de Données

SQL	Structured Query Language
TFLite	TensorFlow Lite
UML	Unified Modeling Language
VARI	Visible Atmospherically Resistant Index
XGB	eXtreme Gradient Boosting

Introduction Générale

1. Contexte général

L'agriculture occupe une place centrale dans l'économie tchadienne : elle contribue à près de 20 % du Produit Intérieur Brut et fait vivre plus de 80 % de la population rurale [1]. Les principales cultures du pays — le mil (*Pennisetum glaucum*), le sorgho (*Sorghum bicolor*), l'arachide (*Arachis hypogaea*), le maïs (*Zea mays*) et le coton (*Gossypium hirsutum*) — sont pratiquées majoritairement en régime pluvial, dans des conditions climatiques difficiles propres aux zones sahélienne et soudanienne : forte variabilité des précipitations, températures élevées et épisodes de sécheresse récurrents.

Parallèlement, l'essor de l'Intelligence Artificielle (IA) ouvre des perspectives inédites pour l'agriculture de précision. Les techniques d'apprentissage automatique permettent aujourd'hui d'exploiter des volumes importants de données climatiques, pédologiques et visuelles afin de produire des recommandations agronomiques personnalisées et en temps réel [2]. Ces avancées représentent une opportunité majeure pour les pays en développement, où l'encadrement agronomique de proximité reste insuffisant.

2. Problématique

Malgré son poids économique, l'agriculture tchadienne demeure peu productive. Les agriculteurs font face à plusieurs obstacles majeurs : la rareté des agronomes qualifiés en zone rurale, l'absence d'accès aux informations météorologiques en temps réel, la difficulté à détecter précocement les maladies des cultures, et une gestion souvent inadaptée des intrants — eau d'irrigation et engrais — entraînant des pertes de rendement et des coûts inutiles.

Face à ce constat, la question centrale de ce travail est la suivante : **comment concevoir un outil d'aide à la décision accessible, précis et adapté aux conditions locales, capable d'accompagner l'agriculteur tchadien dans le suivi quotidien de ses parcelles ?**

Cette problématique se décline en quatre questions opérationnelles :

- Comment prédire le besoin en irrigation d'une parcelle à partir des conditions climatiques locales en temps réel ?
- Comment recommander l'engrais optimal selon les caractéristiques du sol (NPK, pH) et le climat ?

- Comment détecter automatiquement les maladies des cultures à partir d’une photographie de feuille ou d’un indice de végétation ?
- Comment assurer la traçabilité des analyses effectuées afin de permettre un suivi historique des parcelles ?

3. Objectifs du travail

L’objectif général de ce projet est de **concevoir et d’implémenter un cadre décisionnel basé sur l’Intelligence Artificielle pour l’optimisation du suivi parcellaire agricole au Tchad**, matérialisé par une application web dénommée **Agro-IA Tchad**.

Cet objectif général se décompose en cinq objectifs spécifiques :

1. Développer un **module de prédiction du besoin en irrigation** fondé sur l’algorithme Random Forest, entraîné sur des données climatiques réelles de la base NASA POWER couvrant cinq villes tchadiennes sur la période 2015–2023 ;
2. Concevoir un **module de recommandation d’engrais** utilisant l’algorithme XGBoost à partir des paramètres NPK du sol et des conditions climatiques ;
3. Implémenter un **module de détection des maladies** combinant un réseau de neurones convolutif (CNN MobileNetV2) appliqué aux photographies de feuilles, et l’analyse d’indices de végétation NDVI obtenus par satellite Sentinel-2 ou estimés par caméra smartphone ;
4. Mettre en place un **module Historique** assurant la persistance de toutes les analyses dans une base de données SQLite, avec statistiques et export CSV ;
5. **Déployer l’application** sur une plateforme cloud accessible gratuitement depuis tout navigateur web.

4. Motivation et intérêt du sujet

Le choix de ce sujet est motivé par un triple intérêt.

Sur le plan scientifique, ce travail explore l’application concrète de plusieurs familles d’algorithmes d’apprentissage automatique — forêts aléatoires, gradient boosting et réseaux de neurones convolutifs — à des données agricoles réelles et hétérogènes, dans un contexte de ressources limitées (peu de données annotées localement).

Sur le plan socio-économique, un outil d’aide à la décision gratuit et accessible peut contribuer à améliorer les rendements agricoles, à réduire le gaspillage d’eau et d’engrais, et à limiter les pertes de récoltes dues aux maladies, avec un impact direct sur la sécurité alimentaire des ménages ruraux tchadiens.

Sur le plan personnel, ce projet est l’occasion de mettre en œuvre l’ensemble des compétences acquises durant la formation de Licence à l’ENASTIC — programmation, bases de données, génie logiciel et intelligence artificielle — au service d’une problématique nationale concrète.

5. Méthodologie adoptée

Pour atteindre les objectifs fixés, une démarche en quatre phases a été suivie.

La **première phase** a été consacrée à la collecte et au prétraitement des données : extraction de 16 435 observations climatiques depuis l'API NASA POWER, constitution d'un jeu de données de fertilisation de 1 599 observations, et création d'un dataset local de 287 images de feuilles de cultures tchadiennes réparties en 10 classes.

La **deuxième phase** a porté sur l'entraînement et l'optimisation des modèles d'IA sur la plateforme Kaggle (GPU Tesla T4) : Random Forest pour l'irrigation, XGBoost pour la fertilisation et CNN MobileNetV2 par transfert d'apprentissage pour la détection des maladies, avec conversion finale au format TensorFlow Lite.

La **troisième phase** a concerné le développement de l'application web avec le framework Streamlit : intégration des modèles, de l'API météorologique Open-Meteo et de la base de données SQLite pour le module Historique.

Enfin, la **quatrième phase** a été dédiée aux tests, à l'évaluation des performances de chaque module et au déploiement de l'application sur Streamlit Cloud.

6. Structure du mémoire

Outre la présente introduction et la conclusion générale, ce mémoire s'articule autour de quatre chapitres.

Le **Chapitre 1 — Travaux existants** présente l'agriculture de précision, la nature des données agricoles, les techniques d'IA appliquées à l'agriculture, ainsi qu'une analyse des solutions existantes permettant de positionner notre contribution.

Le **Chapitre 2 — Analyse, Conception et Modélisation UML** expose l'analyse du problème, la spécification des besoins, la modélisation UML du système, l'architecture du cadre décisionnel et le modèle de la base de données de l'historique.

Le **Chapitre 3 — Implémentation et Tests** détaille l'environnement de développement, le pré-traitement des données, la réalisation des trois modules d'IA, du module Historique et de l'application web, puis définit le protocole de tests et les métriques d'évaluation.

Le **Chapitre 4 — Résultats et Discussion** présente les résultats obtenus pour chaque module, la validation du module Historique, une discussion générale et le déploiement de l'application.

Le mémoire s'achève par une **conclusion générale** qui dresse la synthèse des travaux, en expose les limites et propose des perspectives d'amélioration.

Chapitre 1

Travaux existants

Introduction

Ce chapitre dresse un état des lieux des connaissances et des technologies sur lesquelles s'appuie notre travail. Il présente d'abord l'agriculture de précision et le suivi parcellaire, puis la nature des données agricoles exploitables. Il expose ensuite les principales techniques d'Intelligence Artificielle appliquées à l'agriculture, avant d'analyser les solutions existantes afin de positionner notre contribution.

1.1 Agriculture de précision et suivi parcellaire

1.1.1 Définition et principes

L'agriculture de précision désigne un ensemble de pratiques et de technologies visant à optimiser la gestion des cultures en tenant compte de la variabilité spatiale et temporelle des parcelles [2]. Plutôt que d'appliquer uniformément les intrants (eau, engrais, produits phytosanitaires), elle propose d'ajuster les interventions aux besoins réels de chaque zone, mesurés à partir de données objectives. Ses bénéfices attendus sont triples : augmentation des rendements, réduction des coûts et préservation de l'environnement.

1.1.2 Le suivi parcellaire

Le suivi parcellaire consiste à observer régulièrement l'état d'une parcelle agricole — humidité du sol, état sanitaire des plantes, stade de développement — afin de déclencher au bon moment les interventions appropriées : irrigation, fertilisation ou traitement. Traditionnellement assuré par l'observation visuelle de l'agriculteur et les conseils d'agronomes, ce suivi peut aujourd'hui être outillé par des capteurs, des stations météorologiques, des images satellitaires et des applications d'aide à la décision.

1.1.3 Contexte agricole du Tchad

Le Tchad dispose d'environ 39 millions d'hectares de terres cultivables, dont une faible fraction seulement est exploitée [1]. L'agriculture y est essentiellement pluviale et familiale, structurée autour de deux grandes zones agro-climatiques : la zone **sahélienne** au centre (N'Djamena, Abéché), caractérisée par une pluviométrie faible et irrégulière (300–600 mm/an), et la zone **soudanienne** au sud (Sarh, Moundou, Bongor), plus arrosée (800–1 200 mm/an). Les cinq cultures retenues dans ce travail — mil, sorgho, arachide, maïs et coton — représentent l'essentiel des superficies cultivées du pays.

1.1.4 Défis du suivi parcellaire en zone sahélienne

Le déploiement du suivi parcellaire au Tchad se heurte à plusieurs obstacles :

- le nombre limité d'agronomes et de techniciens agricoles en zone rurale ;
- l'absence de stations météorologiques de proximité et d'accès aux prévisions localisées ;
- le coût élevé des équipements de mesure (sondes d'humidité, capteurs NPK) ;
- la faible couverture internet et électrique des zones agricoles.

Ces contraintes orientent naturellement vers des solutions logicielles légères, exploitant des données gratuites (satellites, réanalyses climatiques) et accessibles depuis un simple smartphone.

1.2 Données agricoles : nature et hétérogénéité

Les données exploitables pour le suivi parcellaire sont de natures très diverses ; leur hétérogénéité constitue à la fois une richesse et un défi pour les systèmes d'aide à la décision.

1.2.1 Données climatiques

Les variables climatiques — température, humidité relative, précipitations, rayonnement solaire, vitesse du vent et évapotranspiration potentielle (ETP) — conditionnent directement les besoins en eau des cultures. Deux sources gratuites sont particulièrement pertinentes pour le Tchad :

- **NASA POWER** [3] : base de réanalyses climatiques mondiales fournissant des séries journalières historiques depuis 1981, à une résolution de $0,5^\circ \times 0,5^\circ$, utilisée ici pour constituer le jeu d'entraînement du module Irrigation ;
- **Open-Meteo** [4] : API météorologique temps réel et gratuite, sans clé d'authentification, utilisée par l'application pour pré-remplir automatiquement les formulaires.

1.2.2 Données pédologiques

Les propriétés du sol — teneurs en azote (N), phosphore (P) et potassium (K), pH, texture (sableux, limoneux, argileux) et humidité — déterminent la disponibilité des nutriments et le choix des engrais. Ces données proviennent d'analyses de sol en laboratoire ou de kits de terrain.

1.2.3 Données visuelles et télédétection

L'état sanitaire des cultures peut être évalué à partir d'images :

- **Photographies de feuilles** prises au smartphone, analysables par des réseaux de neurones convolutifs pour détecter les symptômes de maladies [5];
- **Images satellitaires** : le programme européen Sentinel-2 [6] fournit gratuitement des images multispectrales à 10 m de résolution, revisitées tous les 5 jours, dont on dérive des indices de végétation;
- **Imagerie aérienne** par drones multispectraux, offrant une résolution centimétrique, mais dont le coût d'acquisition reste élevé pour le contexte tchadien.

1.2.4 L'indice de végétation NDVI

Le NDVI (*Normalized Difference Vegetation Index*) [7] est l'indice le plus utilisé pour quantifier la vigueur de la végétation. Il est calculé à partir des réflectances dans le rouge (RED) et le proche infrarouge (NIR) :

$$\text{NDVI} = \frac{\text{NIR} - \text{RED}}{\text{NIR} + \text{RED}} \quad (1.1)$$

Ses valeurs varient de -1 à $+1$: une végétation dense et saine réfléchit fortement le proche infrarouge ($\text{NDVI} > 0,5$), tandis qu'une végétation stressée ou malade voit son NDVI chuter ($\text{NDVI} < 0,3$).

1.3 Intelligence artificielle appliquée à l'agriculture

1.3.1 Apprentissage automatique classique

Les algorithmes d'apprentissage supervisé classiques excellent sur les données tabulaires (climatiques, pédologiques). Deux familles sont particulièrement adaptées à notre problème :

- **Random Forest** [8] : ensemble d'arbres de décision entraînés sur des sous-échantillons aléatoires, dont la prédiction finale résulte d'un vote majoritaire. Robuste au bruit et peu sensible au surapprentissage, il fournit en outre une mesure d'importance des variables;
- **XGBoost** [9] : implémentation optimisée du *gradient boosting*, construisant séquentiellement des arbres correcteurs d'erreurs. Il domine régulièrement les compétitions de science des données sur données structurées.

1.3.2 Apprentissage profond et vision par ordinateur

Les réseaux de neurones convolutifs (CNN) ont révolutionné la reconnaissance d'images. Appliqués à la santé des plantes, Mohanty *et al.* [5] ont atteint 99,35 % de précision sur le dataset PlantVillage (54 000 images en conditions contrôlées); toutefois, la précision de leur modèle chute à environ 31 %

sur des images de terrain, illustrant le fossé entre conditions de laboratoire et réalité agricole — un point central pour notre travail sur des images tchadiennes réelles.

1.3.3 Transfert d’apprentissage et modèles légers

Le *transfer learning* consiste à réutiliser un réseau pré-entraîné sur un grand corpus générique (ImageNet, 14 millions d’images) et à ne réentraîner que ses couches supérieures sur le jeu de données cible. Cette approche est incontournable lorsque les données annotées sont rares, comme c’est le cas pour les cultures tchadiennes. **MobileNetV2** [10] est une architecture conçue pour les environnements contraints : ses blocs résiduels inversés réduisent drastiquement le nombre de paramètres, et sa conversion au format **TensorFlow Lite** permet un déploiement léger (quelques mégaoctets) adapté au web et au mobile.

1.4 Analyse des solutions existantes et positionnement

1.4.1 Solutions existantes

Plusieurs applications d’agriculture numérique sont déployées en Afrique et dans le monde. Le tableau 1.1 en compare les principales caractéristiques avec notre proposition.

TABLE 1.1 : Comparaison des solutions existantes avec Agro-IA Tchad

Critère	Plantix	Esoko	PlantVillage Nuru	Agro-IA Tchad
Origine	Allemagne	Ghana	International	Tchad
Détection maladies	Oui (photo)	Non	Oui (photo)	Oui (photo + NDVI)
Conseil irrigation	Non	Non	Non	Oui (IA)
Conseil fertilisation	Partiel	Non	Non	Oui (IA)
Historique local	Non	Non	Non	Oui (SQLite)
Cultures tchadiennes	Non	Non	Partiel	Oui (5 cultures)
Météo temps réel	Oui	Oui	Non	Oui (Open-Meteo)
Coût	Gratuit	Abonnement	Gratuit	Gratuit

1.4.2 Limites des solutions existantes

L’analyse du tableau 1.1 révèle trois lacunes majeures pour le contexte tchadien : (i) aucune solution ne couvre simultanément l’irrigation, la fertilisation et la détection des maladies ; (ii) les modèles de reconnaissance sont entraînés sur des cultures et des conditions étrangères, peu représentatives des variétés et symptômes locaux ; (iii) aucune ne propose de traçabilité locale des analyses permettant un suivi historique des parcelles.

1.4.3 Positionnement de notre contribution

Notre système Agro-IA Tchad se positionne comme une réponse intégrée à ces lacunes :

- **Multi-modules** : quatre modules complémentaires (Irrigation, Fertilisation, Détection des maladies, Historique) réunis dans une même application ;
- **Adaptation locale** : modèles entraînés sur des données climatiques tchadiennes (NASA POWER, 5 villes) et sur un dataset d’images collectées au Tchad (287 images, 10 classes) ;
- **Traçabilité** : base SQLite intégrée avec statistiques et export CSV ;
- **Accessibilité** : application web gratuite, déployée sur le cloud, exploitant la météo temps réel.

Conclusion

Ce chapitre a présenté les fondements de l’agriculture de précision, la diversité des données agricoles mobilisables, les techniques d’IA pertinentes — Random Forest, XGBoost et CNN par transfert d’apprentissage — ainsi que les limites des solutions existantes au regard du contexte tchadien. Ce positionnement établi, le chapitre suivant procède à l’analyse des besoins et à la conception détaillée du système Agro-IA Tchad.

Chapitre 2

Analyse, Conception et Modélisation UML

Introduction

Après avoir positionné notre travail par rapport à l'existant, ce chapitre aborde la phase d'analyse et de conception du système Agro-IA Tchad. Il identifie d'abord les acteurs et les problèmes à résoudre, spécifie ensuite les besoins fonctionnels et non fonctionnels, présente la modélisation UML du système, décrit l'architecture du cadre décisionnel et détaille enfin le modèle de la base de données assurant l'historique des analyses.

2.1 Analyse du problème et identification des acteurs

2.1.1 Analyse du problème

L'analyse du contexte agricole tchadien, menée au chapitre précédent, fait ressortir quatre problèmes opérationnels auxquels le système doit répondre :

1. **Gestion approximative de l'irrigation** : en l'absence de données objectives (humidité du sol, ETP, prévisions), l'agriculteur irrigue selon son intuition, avec pour conséquences un gaspillage d'eau ou un stress hydrique des cultures ;
2. **Fertilisation inadaptée** : le choix de l'engrais se fait rarement sur la base d'une analyse du sol, entraînant des dépenses inutiles et un appauvrissement progressif des terres ;
3. **Détection tardive des maladies** : faute de diagnostic précoce, les maladies se propagent et causent des pertes de récolte importantes ;
4. **Absence de traçabilité** : aucune trace des décisions passées n'est conservée, ce qui empêche tout suivi de l'évolution d'une parcelle dans le temps.

2.1.2 Identification des acteurs

Deux acteurs interagissent avec le système :

- **L'Agriculteur (ou agent de terrain)** — acteur principal : il saisit les paramètres de sa parcelle, consulte la météo, lance les analyses et consulte l'historique ;

- **Les services externes** — acteurs secondaires : l'API Open-Meteo qui fournit les données météorologiques en temps réel, et la plateforme Sentinel-2/EO Browser qui fournit les valeurs NDVI.

2.2 Spécification des besoins

2.2.1 Besoins fonctionnels

Le tableau 2.1 récapitule les besoins fonctionnels du système, regroupés par module.

TABLE 2.1 : Besoins fonctionnels du système Agro-IA Tchad

Code	Module	Besoin fonctionnel
BF1	Météo	Afficher la météo temps réel des 5 villes (T°, humidité, pluie, vent, ETP)
BF2	Irrigation	Prédire le besoin en irrigation (OUI/NON) avec dose estimée (mm) et confiance
BF3	Fertilisation	Recommander un engrais (Urée, DAP, NPK 15-15-15, NPK 23-10-5, KCl) avec dose et période d'application
BF4	Maladies	Diagnostiquer l'état d'une culture par photographie de feuille (CNN)
BF5	Maladies	Évaluer l'état d'une parcelle par indice NDVI (satellite ou caméra smartphone)
BF6	Historique	Sauvegarder automatiquement chaque analyse dans la base de données
BF7	Historique	Afficher l'historique, les statistiques et permettre l'export CSV
BF8	Historique	Permettre la suppression de l'historique par module ou en totalité

2.2.2 Besoins non fonctionnels

- **Performance** : temps de réponse d'une analyse inférieur à 3 secondes ;
- **Disponibilité** : accès 24h/24 depuis tout navigateur (ordinateur, tablette, smartphone) ;
- **Utilisabilité** : interface en français, simple, avec pré-remplissage automatique des données météo ;
- **Fiabilité** : accuracy minimale de 75 % pour les modules Irrigation et Fertilisation ;
- **Légèreté** : modèle de détection embarqué au format TFLite (moins de 10 Mo) pour un chargement rapide ;
- **Coût** : utilisation exclusive de services et d'outils gratuits (open source, APIs libres).

2.3 Modélisation UML du système

2.3.1 Diagramme de contexte

La figure 2.1 situe le système dans son environnement : l'agriculteur est l'unique acteur humain ; les services externes (Open-Meteo, Sentinel-2) alimentent le système en données, et la base SQLite assure la persistance locale.

1. DIAGRAMME DE CONTEXTE

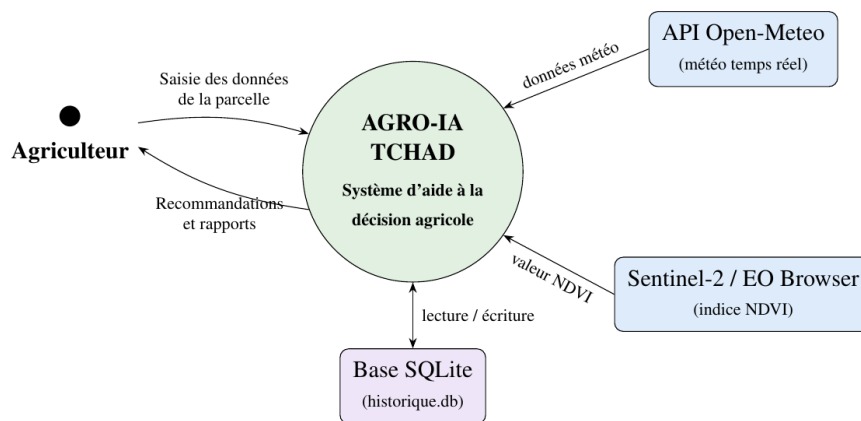


FIGURE 2.1 : Diagramme de contexte du système Agro-IA Tchad

2.3.2 Diagramme de cas d'utilisation

La figure 2.2 présente les cinq cas d'utilisation offerts à l'agriculteur. Les relations « include » matérialisent le recours automatique aux services externes : les modules d'analyse sollicitent l'API Open-Meteo pour pré-remplir les formulaires, et la détection par indice de végétation s'appuie sur les valeurs NDVI relevées sur Sentinel-2.

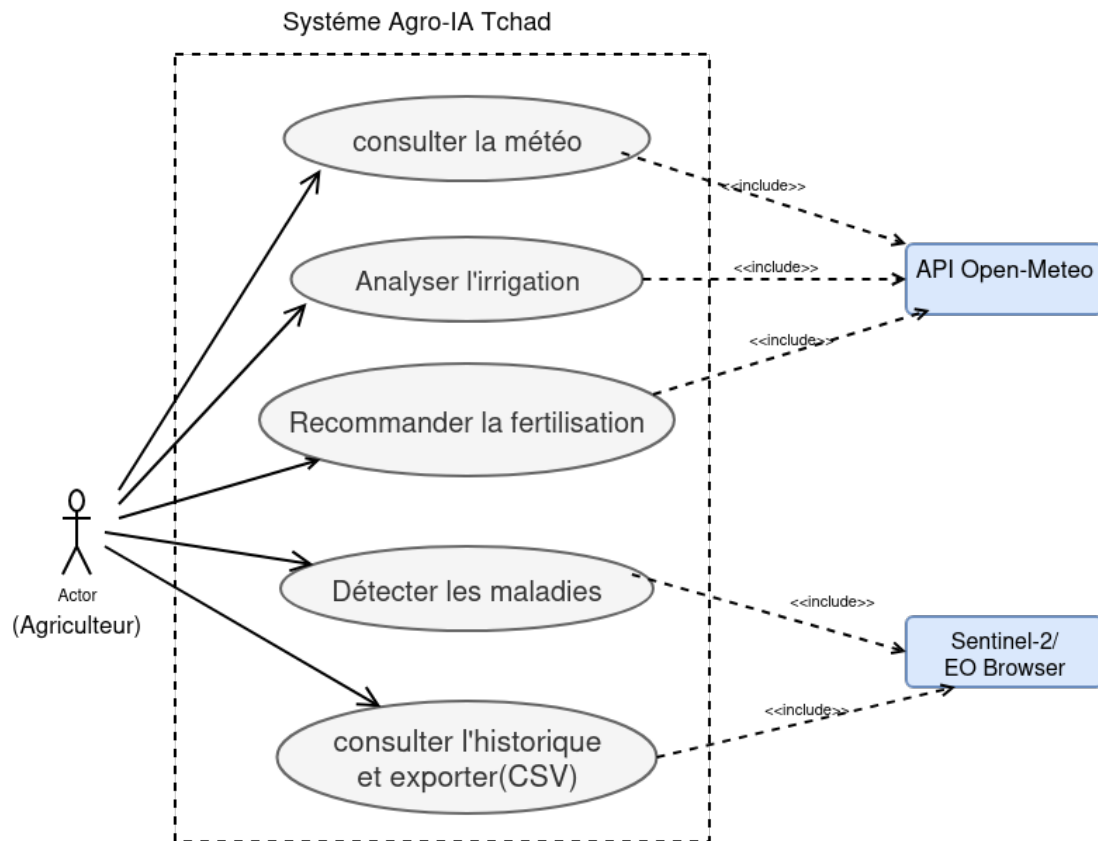


FIGURE 2.2 : Diagramme de cas d'utilisation global avec services externes

2.3.3 Diagramme de séquence — analyse d’une parcelle

La figure 2.3 détaille le scénario nominal d’une analyse : pré-remplissage météo, inférence du modèle, puis sauvegarde automatique dans l’historique avant l’affichage du résultat.

3. DIAGRAMME DE SÉQUENCE : ANALYSE D’UNE PARCELLE

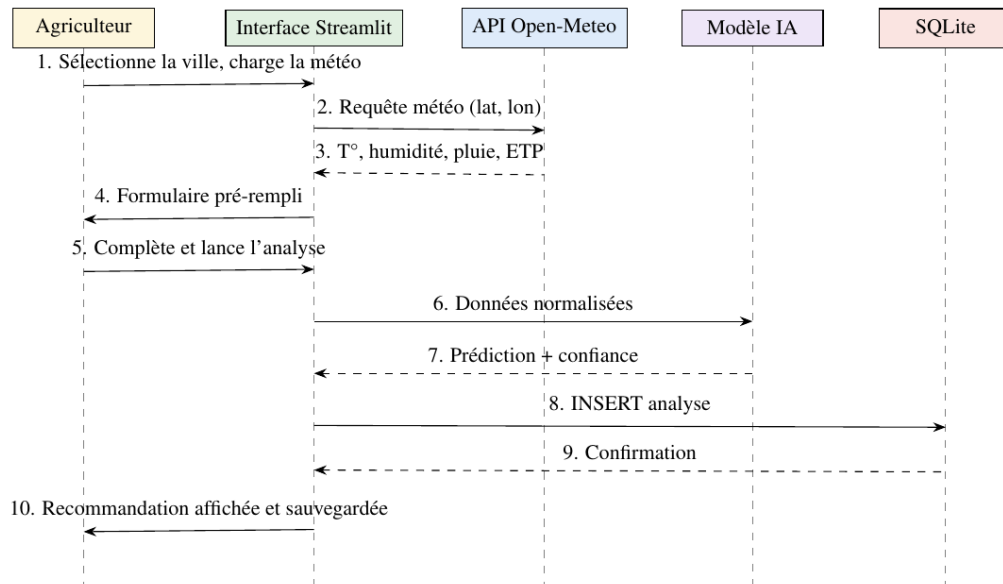


FIGURE 2.3 : Diagramme de séquence — analyse d’une parcelle

2.3.4 Diagramme d’activités — génération d’une recommandation

La figure 2.4 formalise le flot d’activités commun aux trois modules d’analyse, de la saisie des données jusqu’à l’affichage et l’archivage de la recommandation.

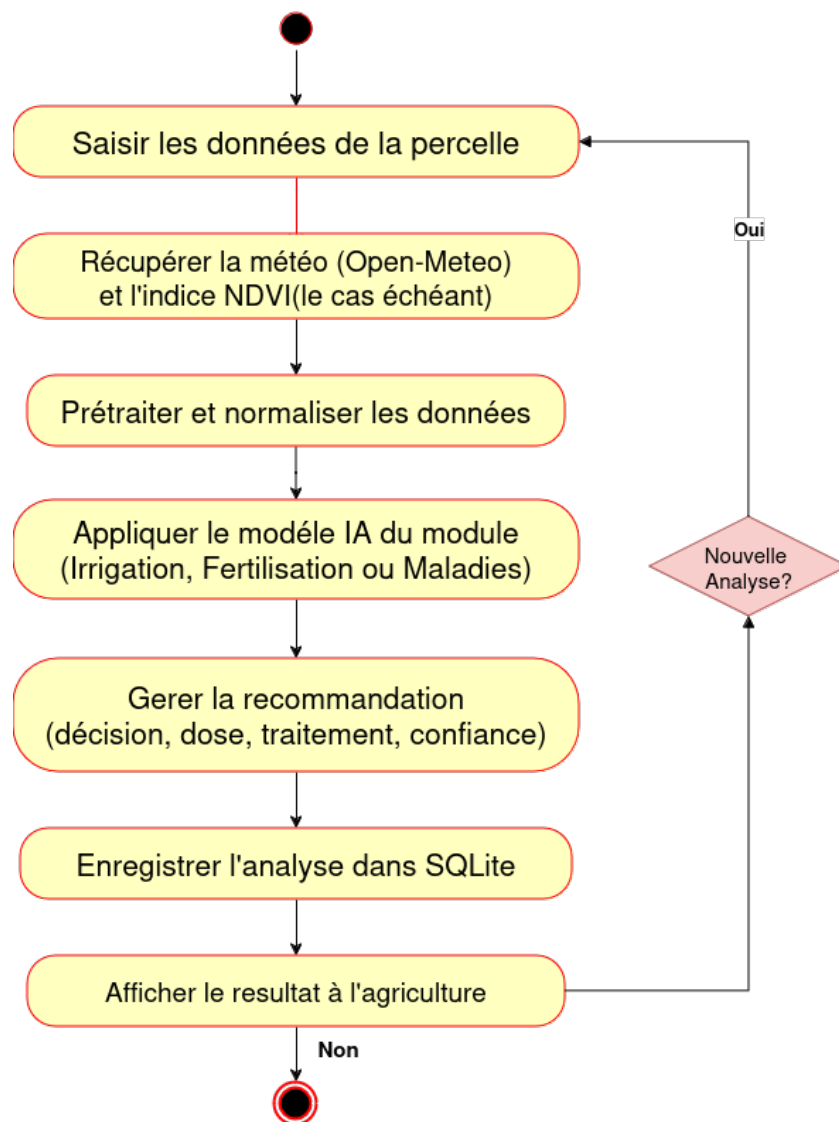


FIGURE 2.4 : Diagramme d'activités — génération d'une recommandation

2.4 Architecture du cadre décisionnel

L'architecture du système est présentée sous deux angles complémentaires : la vue fonctionnelle en pipeline (figure 2.5), qui suit le cheminement des données depuis leurs sources jusqu'à l'agriculteur, et la vue en couches logicielles (figure 2.6).

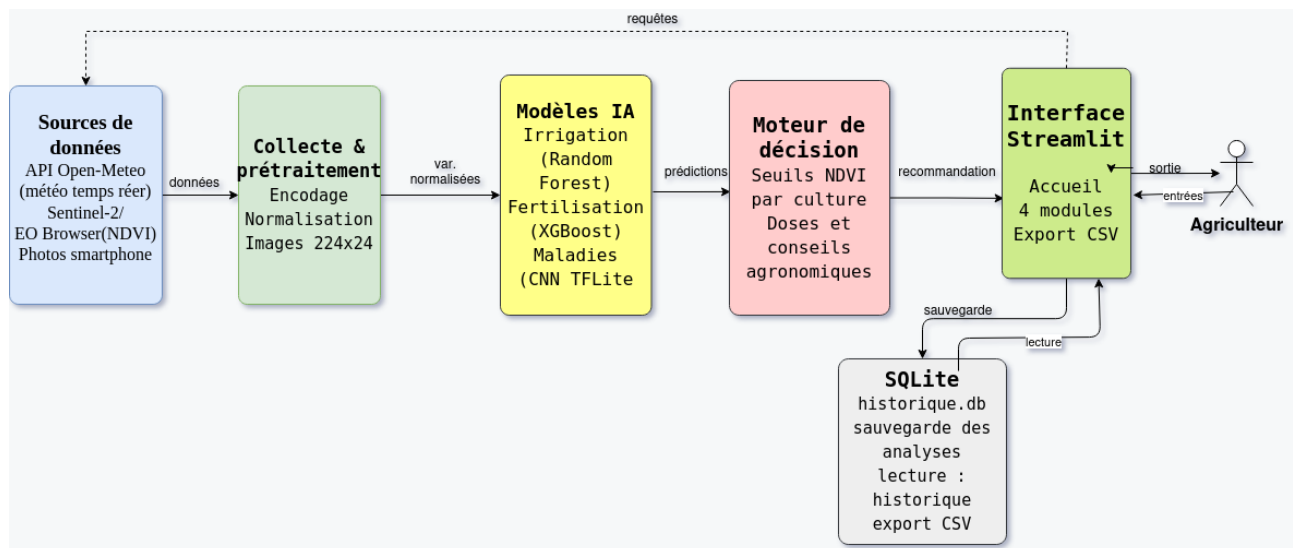


FIGURE 2.5 : Architecture générale du système Agro-IA Tchad

La vue en couches logicielles complète cette lecture :

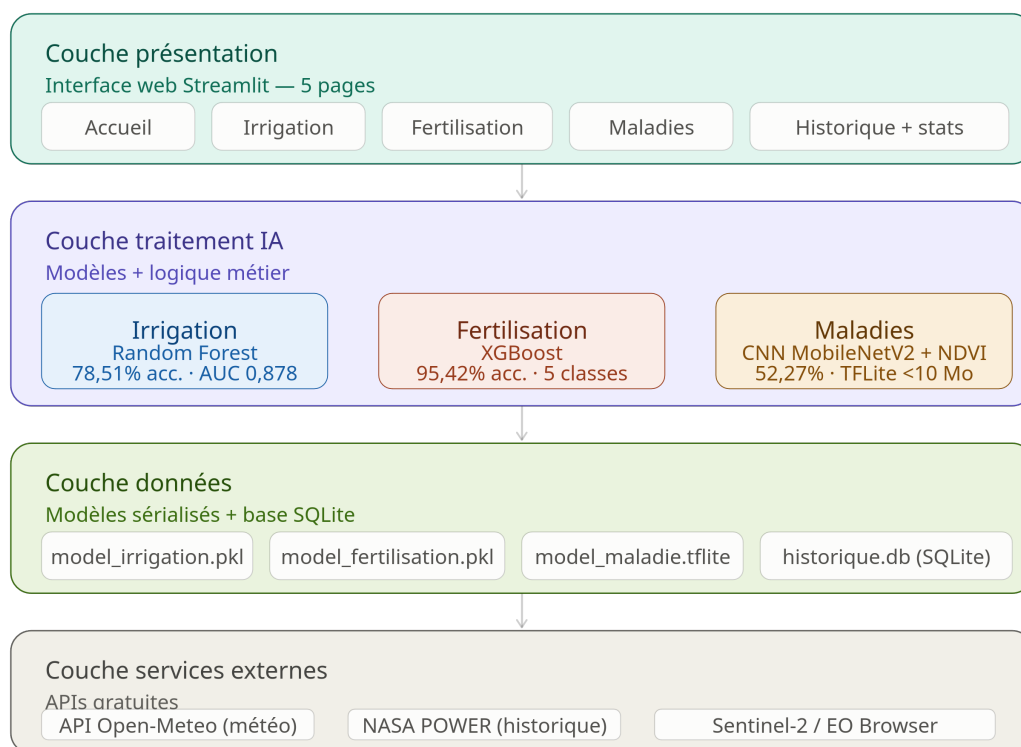


FIGURE 2.6 : Architecture en couches du cadre décisionnel Agro-IA Tchad

- la **couche présentation** regroupe les cinq pages Streamlit (accueil, irrigation, fertilisation, maladies, historique) ;
- la **couche traitement** héberge les modèles d'IA et la logique métier (calcul des doses, seuils NDVI par culture, conseils de traitement) ;

- la **couche données** assure la persistance : modèles entraînés sérialisés et base SQLite de l'historique ;
- la **couche services externes** fournit les données météorologiques temps réel et les indices de végétation.

2.5 Modèle de la base de données de l'historique

Le module Historique répond au besoin de traçabilité (BF6 à BF8). Le SGBD retenu est **SQLite** [11] : embarqué, sans serveur, sans configuration, il est parfaitement adapté à une application mono-utilisateur déployée sur le cloud. La base `historique.db` comporte trois tables indépendantes, une par module d'analyse.

TABLE 2.2 : Structure de la table irrigation

Attribut	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
date_analyse	TEXT	Date et heure de l'analyse
ville, culture, stade	TEXT	Localisation et culture analysée
zone, saison, type_sol	TEXT	Contexte agro-climatique
temperature_moy, humidite_air	REAL	Conditions climatiques
pluie_7j, humidite_sol, etp	REAL	Variables hydriques
resultat	TEXT	Décision : OUI / NON
dose_mm	REAL	Dose d'irrigation estimée (mm)
confiance	REAL	Probabilité associée (%)

TABLE 2.3 : Structure de la table fertilisation

Attribut	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
date_analyse	TEXT	Date et heure de l'analyse
ville, culture, type_sol, zone	TEXT	Contexte de la parcelle
azote_n, phosphore_p, potassium_k	REAL	Analyse NPK du sol
ph_sol	REAL	Acidité du sol
temperature, humidite_air, pluie	REAL	Conditions climatiques
engrais_recommande	TEXT	Engrais prédit par XGBoost
confiance	REAL	Probabilité associée (%)

TABLE 2.4 : Structure de la table maladies

Attribut	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
date_analyse	TEXT	Date et heure de l'analyse
ville, culture	TEXT	Contexte de la parcelle
methode	TEXT	CNN Photo / NDVI satellite / NDVI caméra
resultat, etat	TEXT	Classe prédite et état (Saine, Stress, Malade)
confiance	REAL	Probabilité associée (%)
ndvi	REAL	Valeur NDVI (le cas échéant)
temperature, humidite_air, pluie_7j	REAL	Contexte climatique
traitement	TEXT	Conseil de traitement fourni

Ce modèle volontairement dénormalisé (chaque table est autonome, sans clé étrangère) simplifie les insertions et les lectures, supprime tout risque d'incohérence référentielle et facilite l'export CSV table par table — des choix cohérents avec un usage mono-utilisateur et un volume de données modéré.

Conclusion

Ce chapitre a formalisé la conception du système Agro-IA Tchad : identification des acteurs et des problèmes, spécification de huit besoins fonctionnels couvrant les quatre modules, modélisation UML des interactions, architecture en quatre couches et modèle de données de l'historique. Cette conception constitue le socle de l'implémentation présentée au chapitre suivant.

Chapitre 3

Implémentation et Tests

Introduction

Ce chapitre décrit la réalisation concrète du système conçu au chapitre précédent. Il présente successivement l’environnement de développement, la collecte et le prétraitement des données, l’implémentation des trois modules d’Intelligence Artificielle, du module Historique et de l’application web qui les intègre, puis définit le protocole de tests appliqué à l’ensemble.

3.1 Environnement de développement

Deux environnements complémentaires ont été utilisés : la plateforme cloud Kaggle pour l’entraînement des modèles (accélération GPU), et un poste local sous Ubuntu pour le développement et les tests de l’application. Le tableau 3.1 en résume les caractéristiques.

TABLE 3.1 : Environnements de développement utilisés

Composant	Entraînement (Kaggle)	Développement (local)
Système	Linux (cloud)	Ubuntu 24.04 LTS
Processeur	2 GPU Tesla T4	CPU Intel (HP EliteBook)
Python	3.12	3.12
TensorFlow	2.20 (entraînement)	TFLite Runtime 2.14
Scikit-learn	1.3+	1.3+
XGBoost	2.0+	2.0+
Framework web	—	Streamlit 1.35+
SGBD	—	SQLite 3 (natif Python)
Éditeur	Kaggle Notebook	VS Code
Versionnage	—	Git / GitHub

L’application est organisée selon la structure multi-pages de Streamlit :

```
1 PFE_Agriculture_IA/  
2 | -- app.py                                # Page d'accueil
```

```

3 | -- database.py           # Module Historique (SQLite)
4 | -- meteo_api.py         # Client API Open-Meteo
5 | -- styles.py            # Feuille de style globale
6 | -- requirements.txt     # Dependances Python
7 | -- models/
8 |   | -- model_irrigation.pkl      # Random Forest
9 |   | -- scaler_irrigation.pkl
10 |   | -- model_fertilisation.pkl   # XGBoost
11 |   | -- scaler_fertilisation.pkl
12 |   | -- model_maladie.tflite     # CNN MobileNetV2
13 |   '-- config_maladies.json      # Liste des classes
14 '-- pages/
15 |-- 1_Irrigation.py
16 |-- 2_Fertilisation.py
17 |-- 3_Detection_Maladies.py
18 '-- 4_Historique.py

```

Listing 3.1 – Structure des fichiers de l’application

3.2 Collecte et prétraitement des données

3.2.1 Données d’irrigation — NASA POWER

Les données climatiques ont été extraites de l’API NASA POWER [3] pour les cinq villes cibles (N’Djamena, Abéché, Sarh, Moundou, Bongor) sur la période 2015–2023, soit **16 435 observations journalières**. Quinze variables explicatives ont été retenues : température moyenne et maximale, cumuls de pluie sur 7 et 14 jours, humidité de l’air et du sol, ETP, vent, rayonnement, ainsi que des variables agronomiques encodées (culture, stade phénologique, saison, type de sol, pH, zone climatique). La variable cible binaire (irriguer : OUI/NON) a été construite à partir de règles agronomiques croisant bilan hydrique et stade de la culture.

Le prétraitement a comporté quatre étapes : encodage des variables catégorielles (LabelEncoder), normalisation des variables numériques (StandardScaler), traitement des valeurs manquantes par interpolation, et partitionnement stratifié en ensembles d’entraînement (70 %), de validation (15 %) et de test (15 %).

3.2.2 Données de fertilisation

Le jeu de données de fertilisation combine 99 observations réelles issues d’un dataset public Kaggle et 1 500 observations générées par augmentation cohérente (bruit gaussien contrôlé autour des signatures NPK de chaque engrais), soit **1 599 observations** couvrant cinq classes d’engrais : Urée, DAP, NPK 15-15-15, NPK 23-10-5 et KCl.

3.2.3 Dataset local d'images — maladies

Un dataset de **287 images** de feuilles de cultures tchadiennes a été constitué, réparti en **10 classes** (5 cultures $\times$ 2 états : saine/malade), comme détaillé en annexe A. Les images, prises en conditions réelles de terrain, ont été redimensionnées en 224×224 pixels et normalisées dans $[0, 1]$. Pour compenser la petite taille du dataset, une augmentation de données a été appliquée à l'entraînement : rotations ($\pm 30^\circ$), symétries horizontale et verticale, zoom ($\pm 20\%$) et variations de luminosité (0,65 à 1,35).

3.3 Module IA — Irrigation

3.3.1 Choix de l'algorithme Random Forest

Le Random Forest [8] agrège les prédictions de T arbres de décision entraînés sur des échantillons *bootstrap* ; la classe finale résulte du vote majoritaire :

$$\hat{y} = \arg \max_{c \in \{0,1\}} \sum_{t=1}^T \mathbf{1}[h_t(\mathbf{x}) = c] \quad (3.1)$$

où $h_t(\mathbf{x})$ désigne la prédiction du t -ième arbre pour l'observation $\mathbf{x}$. Cet algorithme a été retenu pour sa robustesse au bruit, sa résistance au surapprentissage et sa capacité à mesurer l'importance des variables.

3.3.2 Entraînement et hyperparamètres

Les hyperparamètres ont été optimisés par validation croisée à 5 plis. La configuration retenue est : 100 arbres (`n_estimators`), profondeur non limitée (`max_depth=None`), pondération équilibrée des classes (`class_weight=balanced`) et graine fixée (`random_state=42`) pour la reproductibilité.

3.3.3 Intégration dans l'application

Le modèle et son normaliseur, sérialisés avec `joblib`, sont chargés une seule fois grâce au cache de Streamlit :

```

1  import joblib, numpy as np
2
3  model = joblib.load("models/model_irrigation.pkl")
4  scaler = joblib.load("models/scaler_irrigation.pkl")
5
6  def predire_irrigation(features):
7      X = scaler.transform(np.array([features]))
8      y = model.predict(X)[0]
9      p = model.predict_proba(X)[0]
10     return {"decision": "OUI" if y == 1 else "NON",
11            "confiance": round(p[y] * 100, 1)}

```

Listing 3.2 – Chargement et prédiction du module Irrigation

3.4 Module IA — Fertilisation

3.4.1 Choix de l’algorithme XGBoost

XGBoost [9] construit séquentiellement des arbres correcteurs en minimisant une fonction objectif régularisée :

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (3.2)$$

où l est la perte (entropie croisée multi-classes) et $\Omega(f_k)$ pénalise la complexité de l’arbre k , limitant le surapprentissage — un atout essentiel sur notre jeu de données de taille modeste.

3.4.2 Entrées et sorties du module

Le modèle prend en entrée onze variables (N, P, K, pH, température, humidités de l’air et du sol, pluie, culture, type de sol, zone) et prédit l’un des cinq engrais. La recommandation affichée est enrichie d’une dose indicative (kg/ha) et de la période d’application, issues d’une table de correspondance agronomique.

3.5 Module IA — Détection des maladies

3.5.1 Architecture CNN MobileNetV2 et transfert d’apprentissage

Le réseau retenu est MobileNetV2 [10] pré-entraîné sur ImageNet, surmonté d’une tête de classification adaptée : GlobalAveragePooling, couche dense de 256 neurones (ReLU) avec Batch-Normalization et Dropout (0,5), couche dense de 128 neurones (ReLU) avec BatchNormalization et Dropout (0,4), et couche de sortie Softmax à 10 classes.

L’entraînement, mené sur Kaggle (2 GPU T4), a suivi une stratégie en deux phases : (i) base gelée, seule la tête est entraînée (taux d’apprentissage 10^{-3} , arrêt précoce avec patience de 10 époques); (ii) *fine-tuning* des 30 dernières couches de la base (taux réduit à 10^{-4}).

3.5.2 Conversion au format TensorFlow Lite

Pour garantir un déploiement léger et compatible avec l’environnement cloud, le modèle Keras a été converti au format TFLite [12] avec quantification par défaut, ramenant sa taille à moins de 10 Mo. L’inférence dans l’application s’effectue via la bibliothèque légère *tflite-runtime*, sans dépendance à TensorFlow complet.

3.5.3 Analyse NDVI : satellite et caméra smartphone

En complément du diagnostic par photographie, le module propose une évaluation par indice de végétation selon deux sources :

- **Satellite Sentinel-2** [6] : l'utilisateur relève la valeur NDVI de sa parcelle sur la plateforme gratuite EO Browser et la saisit dans l'application, qui l'interprète selon des seuils spécifiques à chaque culture ;
- **Caméra smartphone** : à défaut de capteur proche infrarouge, l'indice VARI est calculé sur les canaux visibles de la photographie :

$$\text{VARI} = \frac{G - R}{G + R - B} \quad (3.3)$$

puis converti en estimation du NDVI par la relation empirique :

$$\text{NDVI}_{\text{estimé}} = 0,18 + 0,73 \times \text{VARI} \quad (3.4)$$

Le diagnostic final croise l'indice obtenu avec les conditions climatiques (température, humidité, pluie) pour classer la parcelle en trois états : saine, en stress ou malade, assortis d'un conseil de traitement.

3.6 Module Historique — Persistance SQLite et export CSV

3.6.1 Implémentation de la couche d'accès aux données

Le module `database.py` centralise toutes les opérations sur la base `historique.db`, conformément au modèle défini à la section 2.5. À l'initialisation, les trois tables sont créées si elles n'existent pas (`CREATE TABLE IF NOT EXISTS`). Chaque module d'analyse appelle ensuite sa fonction de sauvegarde dédiée :

```

1  import sqlite3
2  from datetime import datetime
3
4  def sauvegarder_irrigation(ville, culture, resultat,
5  dose_mm, confiance):
6  conn = sqlite3.connect("historique.db")
7  conn.execute("""
8  INSERT INTO irrigation
9  (date_analyse, ville, culture,
10 resultat, dose_mm, confiance)
11 VALUES (?, ?, ?, ?, ?, ?)""",
12 (datetime.now().strftime("%Y-%m-%d %H:%M"),
13 ville, culture, resultat, dose_mm, confiance))
14 conn.commit()
```

15

```
conn.close()
```

Listing 3.3 – Sauvegarde d’une analyse d’irrigation dans SQLite

L’utilisation de requêtes paramétrées (?) protège la base contre les injections SQL.

3.6.2 Consultation, statistiques et export

La page Historique agrège les trois tables et offre :

- une **vue chronologique unifiée** des analyses (module, culture, ville, résultat, confiance);
- des **statistiques globales** : nombre d’analyses par module, culture la plus analysée, zone à risque (ville comptant le plus de diagnostics « malade »), taux d’irrigation recommandée et taux de maladies détectées;
- l’**export CSV** de chaque table ou de l’historique complet, généré en mémoire et proposé au téléchargement;
- la **purge sélective** de l’historique, par module ou en totalité.

3.7 Développement de l’application web

3.7.1 Framework Streamlit

L’interface a été développée avec Streamlit [13], framework Python permettant de construire des applications web de données sans développement front-end distinct. L’application compte cinq pages : l’accueil (métriques des modèles et statistiques de l’historique) et les quatre modules. Les modèles sont chargés une seule fois par session grâce au décorateur `@st.cache_resource`, garantissant des temps de réponse inférieurs à la contrainte des 3 secondes.

3.7.2 Intégration de la météo temps réel

Le module `meteo_api.py` interroge l’API Open-Meteo [4] pour les cinq villes couvertes : conditions courantes (température, humidité, vent), cumuls de pluie sur 7 et 14 jours, ETP et humidité du sol. Les réponses sont mises en cache 30 minutes (`@st.cache_data`) afin de limiter les appels réseau. Ces valeurs pré-remplissent automatiquement les formulaires d’analyse, réduisant la saisie manuelle de l’agriculteur.

3.7.3 Interface et expérience utilisateur

Une feuille de style commune (`styles.py`) assure la cohérence visuelle : thème aux couleurs de la végétation, cartes de résultats colorées selon la décision (bleu : irrigation recommandée; vert : parcelle saine; orange : stress; rouge : maladie), et affichage systématique de la confiance de la prédiction pour une décision éclairée.

3.8 Protocole de tests

3.8.1 Métriques d'évaluation des modèles

Les modèles de classification sont évalués à partir de la matrice de confusion (vrais/faux positifs TP , FP ; vrais/faux négatifs TN , FN) au moyen des métriques usuelles :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.5)$$

$$\text{Précision} = \frac{TP}{TP + FP} \quad \text{Rappel} = \frac{TP}{TP + FN} \quad (3.6)$$

$$F1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (3.7)$$

S'y ajoutent l'**AUC-ROC** (aire sous la courbe ROC, mesurant le pouvoir discriminant indépendamment du seuil de décision) et le **score de validation croisée** à $k = 5$ plis, qui apprécie la stabilité du modèle.

3.8.2 Démarche de validation des modèles

Chaque jeu de données a été partitionné de façon stratifiée en entraînement (70 %), validation (15 %) et test (15 %). L'optimisation des hyperparamètres s'est appuyée exclusivement sur les ensembles d'entraînement et de validation ; les performances rapportées au chapitre suivant sont mesurées sur l'ensemble de test, jamais vu pendant l'entraînement.

3.8.3 Plan de tests fonctionnels de l'application

Au-delà des modèles, l'application elle-même a été soumise à un plan de tests fonctionnels couvrant : le pré-remplissage météo (BF1), le déroulement complet d'une analyse pour chacun des trois modules d'IA (BF2 à BF5), et le module Historique — sauvegarde automatique, consultation unifiée, exactitude des statistiques, export CSV et purge sélective (BF6 à BF8). Ces tests ont été exécutés en local puis rejoués sur l'environnement de production ; leurs résultats sont présentés au chapitre 4.

Conclusion

Ce chapitre a détaillé l'implémentation complète du système : préparation de trois jeux de données, entraînement des modèles Random Forest, XGBoost et CNN MobileNetV2 (converti en TFLite), réalisation du module Historique fondé sur SQLite avec export CSV, et intégration de l'ensemble dans une application web Streamlit alimentée par la météo temps réel. Un protocole de tests rigoureux — métriques, partitionnement stratifié et plan de tests fonctionnels — a été défini. Le chapitre suivant en présente les résultats et les discute.

Chapitre 4

Résultats et Discussion

Introduction

Ce dernier chapitre présente les résultats obtenus en appliquant le protocole de tests défini à la section 3.8. Les performances de chacun des trois modules d’IA sont d’abord exposées et analysées, puis les tests fonctionnels du module Historique sont restitués. Une discussion générale met ensuite ces résultats en perspective, avant la présentation du déploiement de l’application.

4.1 Résultats du module Irrigation

Le tableau 4.1 présente les performances du Random Forest sur l’ensemble de test.

TABLE 4.1 : Performances du module Irrigation (Random Forest)

Métrique	Description	Valeur
Accuracy	Taux de prédictions correctes	78,51 %
Précision	Fiabilité des alertes d’irrigation	68,59 %
Rappel	Besoins d’irrigation détectés	77,03 %
F1-Score	Équilibre précision/rappel	72,57 %
AUC-ROC	Pouvoir discriminant	0,8782
CV (k=5)	Score moyen en validation croisée	78,87 %
Observations	Jeu NASA POWER (5 villes)	16 435
Arbres	n_estimators	100

L’AUC-ROC de 0,8782 traduit une bonne capacité à distinguer les situations nécessitant une irrigation. Le rappel de 77 % — privilégié par la pondération équilibrée des classes — limite les faux négatifs, c’est-à-dire les stress hydriques non détectés, plus dommageables pour la culture qu’une irrigation superflue. L’écart minime entre accuracy de test (78,51 %) et score de validation croisée (78,87 %) confirme la stabilité du modèle.

L'analyse de l'importance des variables place en tête l'humidité de l'air (0,124), le cumul de pluie sur 7 jours (0,117) et le cumul sur 14 jours (0,114) : la décision d'irriguer est donc principalement gouvernée par l'état hydrique récent de l'atmosphère et du sol, ce qui est agronomiquement cohérent.

4.2 Résultats du module Fertilisation

TABLE 4.2 : Performances du module Fertilisation (XGBoost)

Métrique	Description	Valeur
Accuracy	Taux de recommandations correctes	95,42 %
F1-Score macro	Moyenne des F1 des 5 classes	95,47 %
CV (k=5)	Score moyen en validation croisée	97,23 %
Écart train/val	Contrôle du surapprentissage	3,75 %
Observations	Kaggle réel + augmentation	1 599
Classes	Types d'engrais	5

Les excellentes performances (tableau 4.2) s'expliquent par la structure du problème : chaque engrais possède une signature NPK distinctive que le modèle apprend aisément. L'analyse d'importance le confirme : le phosphore domine (0,439), suivi de l'azote (0,336) et du potassium (0,050) — le phosphore est en effet le principal discriminant du DAP (18-46-0) face à l'urée (46-0-0) et au KCl (0-0-60). Le faible écart entraînement/validation (3,75 %) écarte l'hypothèse d'un surapprentissage malgré la part de données augmentées.

4.3 Résultats du module Détection des maladies

TABLE 4.3 : Performances du CNN MobileNetV2 (dataset tchadien)

Métrique	Description	Valeur
Val. accuracy (phase 1)	Tête seule, base gelée	65,12 %
Val. accuracy (phase 2)	Après <i>fine-tuning</i>	60,47 %
Accuracy (test)	Ensemble de test final	52,27 %
F1-Score macro	Moyenne des 10 classes	47,74 %
Images	Dataset local tchadien	287
Classes	5 cultures × 2 états	10
Modèle TFLite	Taille après quantification	< 10 Mo

L'accuracy de test de 52,27 % — soit plus de cinq fois le niveau du hasard (10 % pour 10 classes) — doit être interprétée au regard des contraintes du dataset : environ 28 images par classe, très en deçà

des standards du domaine, et des prises de vue en conditions réelles (fonds complexes, éclairages variables). Cette lecture est confortée par la littérature : le modèle de Mohanty *et al.* [5], qui atteint 99,35 % sur les images de laboratoire de PlantVillage, chute à environ 31 % sur des images de terrain. Notre résultat, obtenu directement sur des images de terrain tchadiennes, constitue ainsi une base honorable, que l’enrichissement du dataset devrait porter à 70–80 %. Les classes les mieux reconnues sont « Arachide saine », « Maïs sain » et « Maïs malade » ; les confusions se concentrent sur les céréales à morphologie proche (mil et sorgho), dont les feuilles sont difficiles à distinguer même pour un œil non expert.

4.4 Résultats des tests du module Historique

Le plan de tests fonctionnels défini à la section 3.8 a été exécuté ; le tableau 4.4 en présente la synthèse.

TABLE 4.4 : Résultats des tests fonctionnels du module Historique

Cas de test	Résultat attendu	Verdict
Sauvegarde après analyse	Une ligne insérée dans la table du module concerné, horodatée	Validé
Consultation unifiée	Les analyses des trois tables apparaissent triées par date décroissante	Validé
Statistiques	Compteurs par module, culture la plus analysée, zone à risque et taux calculés exacts	Validé
Export CSV	Fichiers téléchargés lisibles dans un tableur, encodage UTF-8, toutes colonnes présentes	Validé
Purge sélective	Suppression par module sans affecter les autres tables ; purge totale effective	Validé
Persistance	Les données survivent au redémarrage local de l’application	Validé

Une limite est à signaler : sur l’hébergement cloud gratuit, le système de fichiers est éphémère — la base est réinitialisée à chaque redémarrage du conteneur. L’export CSV prend alors tout son sens, en permettant à l’utilisateur de conserver localement ses données ; en local, la persistance est totale.

4.5 Discussion générale des résultats

TABLE 4.5 : Synthèse des performances des trois modules d’IA

Module	Algorithme	Accuracy	F1-Score
Irrigation	Random Forest	78,51 %	72,57 %
Fertilisation	XGBoost	95,42 %	95,47 %
Maladies	CNN MobileNetV2	52,27 %	47,74 %

Trois enseignements se dégagent du tableau 4.5. D’abord, l’adéquation entre algorithme et nature des données est déterminante : sur données tabulaires abondantes (irrigation) ou bien structurées (fertilisation), les méthodes d’ensemble atteignent des niveaux directement exploitables ; sur images de terrain en faible quantité, l’apprentissage profond reste pénalisé malgré le transfert d’apprentissage. Ensuite, l’objectif de fiabilité fixé au chapitre 2 ($\text{accuracy} \geq 75\%$) est atteint pour l’irrigation et largement dépassé pour la fertilisation. Enfin, la principale marge de progression — le module maladies — relève de la donnée plus que du modèle : c’est l’enrichissement du dataset local qui conditionnera le saut de performance.

4.6 Déploiement de l’application

4.6.1 Déploiement local

En environnement local (Ubuntu 24.04, Python 3.12), l’application se lance par la commande `streamlit run app.py` après installation des dépendances du fichier `requirements.txt` dans un environnement virtuel. L’ensemble des modules, y compris le CNN via `tflite-runtime`, y est opérationnel.

4.6.2 Déploiement sur Streamlit Cloud

Le code source est hébergé sur GitHub et déployé en continu sur Streamlit Cloud [14] : chaque `git push` déclenche la reconstruction de l’application. Deux adaptations ont été nécessaires : le verrouillage de la version de Python à 3.11 (fichier `.python-version`), et le remplacement de TensorFlow — indisponible pour les versions récentes de Python de la plateforme — par la bibliothèque légère `tflite-runtime` pour l’inférence du CNN. L’application est accessible publiquement à l’adresse :

<https://agro-ia-tchad.streamlit.app>

Conclusion

Les résultats confirment l’atteinte des objectifs : le module Fertilisation excelle (95,42 %), le module Irrigation dépasse le seuil de fiabilité fixé (78,51 %, AUC 0,878), le module Maladies fournit une base prometteuse sur images de terrain réelles (52,27 %), et le module Historique remplit intégralement son cahier des charges de traçabilité et d’export. L’application, déployée et accessible en ligne, matérialise l’aboutissement opérationnel du projet. La conclusion générale qui suit en dresse le bilan et trace les perspectives.

Conclusion Générale

Synthèse des travaux

Ce mémoire avait pour ambition de répondre à une question concrète : comment offrir à l’agriculteur tchadien un outil d’aide à la décision accessible, fiable et adapté aux réalités locales ? La réponse apportée est le système **Agro-IA Tchad**, un cadre décisionnel complet articulé autour de quatre modules.

Le **module Irrigation**, fondé sur un Random Forest entraîné sur 16 435 observations climatiques réelles de la base NASA POWER (cinq villes, 2015–2023), prédit le besoin en eau avec une accuracy de 78,51 % et un AUC-ROC de 0,8782, dépassant le seuil de fiabilité fixé lors de la spécification des besoins. Le **module Fertilisation**, s’appuyant sur XGBoost et 1 599 observations, recommande l’engrais optimal avec une accuracy remarquable de 95,42 %. Le **module Détection des maladies** associe un CNN MobileNetV2 — entraîné par transfert d’apprentissage sur un dataset original de 287 images de cultures tchadiennes, une contribution en soi — à l’analyse d’indices NDVI issus de Sentinel-2 ou estimés par caméra smartphone ; son accuracy de 52,27 % sur images de terrain constitue une base solide au regard de la littérature. Enfin, le **module Historique** garantit la traçabilité de toutes les analyses via une base SQLite, avec statistiques, export CSV et purge sélective, l’ensemble des tests fonctionnels ayant été validés.

L’application web qui intègre ces modules, développée avec Streamlit et alimentée en temps réel par l’API Open-Meteo, est déployée sur Streamlit Cloud et accessible gratuitement depuis tout navigateur. Les objectifs fixés en introduction sont ainsi atteints.

Limites rencontrées

Ce travail comporte néanmoins des limites qu’il convient de souligner avec honnêteté. La première tient à la **taille du dataset d’images** : avec environ 28 images par classe, le module de détection des maladies ne peut rivaliser avec les systèmes entraînés sur des dizaines de milliers d’images ; les confusions entre céréales morphologiquement proches (mil, sorgho) en sont la conséquence directe. La deuxième concerne la **persistance sur l’hébergement cloud gratuit**, dont le système de fichiers éphémère réinitialise la base SQLite à chaque redémarrage — l’export CSV atténue cette contrainte sans la supprimer. La troisième est l’**absence de validation agronomique de terrain** : les recommandations n’ont pas encore été confrontées à des campagnes réelles menées avec des agriculteurs et des

agronomes de l'ITRAD. Enfin, la **dépendance à la connectivité internet** limite l'usage dans les zones rurales les moins couvertes.

Perspectives d'amélioration

Les perspectives ouvertes par ce travail s'organisent en trois horizons.

À **court terme**, l'enrichissement du dataset local à 100–200 images par classe — par des campagnes de collecte photographique dans les zones de production — devrait porter l'accuracy du module maladies à 70–80 %. L'intégration de l'API Sentinel Hub permettrait en outre de récupérer automatiquement le NDVI d'une parcelle géolocalisée, supprimant la saisie manuelle.

À **moyen terme**, deux évolutions majeures sont envisagées : d'une part le passage à une architecture client-serveur (Django, PostgreSQL) offrant des comptes utilisateurs et une persistance pérenne de l'historique ; d'autre part l'intégration de la **détection des maladies par drone multispectral**, qui permettrait de cartographier l'état sanitaire de parcelles entières à résolution centimétrique — capteurs proche infrarouge embarqués, calcul de cartes NDVI/NDRE complètes et localisation précise des foyers d'infection — là où la version actuelle repose sur des observations ponctuelles (photo de feuille, pixel satellite).

À **long terme**, le système pourrait évoluer vers une plateforme nationale de suivi agricole : application mobile fonctionnant hors-ligne avec synchronisation différée, interfaces en langues locales, intégration de capteurs IoT in situ (humidité du sol, NPK) et tableaux de bord agrégés à destination des services agricoles de l'État et de l'ITRAD.

En définitive, Agro-IA Tchad démontre qu'une intelligence artificielle frugale — données ouvertes, modèles légers, outils gratuits — peut produire un outil opérationnel au service de l'agriculture sahélienne, et pose les fondations techniques d'un système d'information agricole à l'échelle du pays.

Bibliographie

- [1] BANQUE MONDIALE, *Tchad – Vue d’ensemble et données agricoles*, 2023. adresse : <https://donnees.banquemondiale.org/pays/TD> (visité le 15/05/2026).
- [2] K. G. LIAKOS, P. BUSATO, D. MOSHOU, S. PEARSON et D. BOCHTIS, « Machine Learning in Agriculture : A Review, » *Sensors*, t. 18, n° 8, p. 2674, 2018. DOI : [10.3390/s18082674](https://doi.org/10.3390/s18082674). adresse : <https://www.mdpi.com/1424-8220/18/8/2674>.
- [3] P. W. STACKHOUSE, T. ZHANG, D. WESTBERG et al., *NASA POWER : Prediction Of Worldwide Energy Resources – Agroclimatology Methodology*, NASA Langley Research Center, 2019. adresse : <https://power.larc.nasa.gov> (visité le 25/05/2026).
- [4] P. ZIPPENFENIG, *Open-Meteo.com Weather API*, 2023. DOI : [10.5281/zenodo.7970649](https://doi.org/10.5281/zenodo.7970649). adresse : <https://open-meteo.com> (visité le 10/06/2026).
- [5] S. P. MOHANTY, D. P. HUGHES et M. SALATHÉ, « Using Deep Learning for Image-Based Plant Disease Detection, » *Frontiers in Plant Science*, t. 7, p. 1419, 2016. DOI : [10.3389/fpls.2016.01419](https://doi.org/10.3389/fpls.2016.01419). adresse : <https://www.frontiersin.org/articles/10.3389/fpls.2016.01419/full>.
- [6] EUROPEAN SPACE AGENCY, *Sentinel-2 User Handbook*, 2015. adresse : <https://sentinel.esa.int/web/sentinel/user-guides/sentinel-2-msi> (visité le 18/05/2025).
- [7] J. W. ROUSE, R. H. HAAS, J. A. SCHELL et D. W. DEERING, « Monitoring vegetation systems in the Great Plains with ERTS, » *NASA Special Publication*, t. 351, p. 309-317, 1974. adresse : <https://ntrs.nasa.gov/citations/19740022614>.
- [8] L. BREIMAN, « Random Forests, » *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. DOI : [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324). adresse : <https://link.springer.com/article/10.1023/A:1010933404324>.
- [9] T. CHEN et C. GUESTRIN, « XGBoost : A Scalable Tree Boosting System, » in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, p. 785-794. DOI : [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785). adresse : <https://arxiv.org/abs/1603.02754>.
- [10] M. SANDLER, A. HOWARD, M. ZHU, A. ZHMOGINOV et L.-C. CHEN, « MobileNetV2 : Inverted Residuals and Linear Bottlenecks, » in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, p. 4510-4520. DOI : [10.1109/CVPR.2018.00474](https://doi.org/10.1109/CVPR.2018.00474). adresse : <https://arxiv.org/abs/1801.04381>.

- [11] SQLITE CONSORTIUM, *SQLite : Small. Fast. Reliable. Choose any three.* 2024. adresse : <https://sqlite.org> (visité le 20/06/2025).
- [12] GOOGLE BRAIN TEAM, *TensorFlow Lite : Deploy machine learning models on mobile and edge devices*, 2023. adresse : <https://www.tensorflow.org/lite> (visité le 05/06/2025).
- [13] STREAMLIT INC., *Streamlit : A faster way to build and share data apps*, 2024. adresse : <https://streamlit.io> (visité le 15/06/2025).
- [14] I. H. TOURABI, *Agro-IA Tchad – Application web et code source*, Application en ligne : agro-ia-tchad.streamlit.app, 2026. adresse : https://github.com/tourabiissakhissein33-dev/PFE_Agriculture_IA (visité le 01/07/2026).

Annexe A

Jeux de données utilisés

A.1 — Dataset Irrigation (NASA POWER)

TABLE A.1 : Caractéristiques du dataset d’irrigation

Caractéristique	Valeur
Source	NASA POWER (power.larc.nasa.gov)
Période	2015–2023
Observations	16 435
Variables	15 explicatives + 1 cible binaire
Villes	N’Djamena, Abéché, Sarh, Moundou, Bongor
Cultures	Mil, Sorgho, Arachide, Maïs, Coton
Cible	Besoin en irrigation (OUI / NON)

A.2 — Dataset Fertilisation

TABLE A.2 : Caractéristiques du dataset de fertilisation

Caractéristique	Valeur
Sources	Dataset public Kaggle (99 obs. réelles) + augmentation cohérente (1 500 obs.)
Observations	1 599
Variables	N, P, K, pH, température, humidités, pluie, culture, type de sol, zone
Classes (5)	Urée, DAP, NPK 15-15-15, NPK 23-10-5, KCl

A.3 — Dataset local d'images (maladies)

TABLE A.3 : Distribution du dataset local tchadien (287 images)

Classe	Description	Images
Arachide malade	Feuilles d'arachide malades	28
Arachide saine	Feuilles d'arachide saines	40
Coton malade	Feuilles de coton malades	30
Coton sain	Feuilles de coton saines	26
Maïs malade	Feuilles de maïs malades	27
Maïs sain	Feuilles de maïs saines	27
Mil malade	Feuilles de mil malades	35
Mil sain	Feuilles de mil saines	23
Sorgho malade	Feuilles de sorgho malades	27
Sorgho sain	Feuilles de sorgho saines	24
Total	10 classes	287

Annexe B

Codes sources principaux

B.1 — Inférence du CNN au format TFLite

```
1  import numpy as np, json
2  import tflite_runtime.interpreter as tflite
3
4  interpreter = tflite.Interpreter(
5  model_path="models/model_maladie.tflite")
6  interpreter.allocate_tensors()
7  classes = json.load(
8  open("models/config_maladies.json"))["classes"]
9
10 def predire(image_pil):
11     img = image_pil.convert("RGB").resize((224, 224))
12     arr = np.expand_dims(
13     np.array(img, dtype=np.float32) / 255.0, 0)
14     inp = interpreter.get_input_details()
15     out = interpreter.get_output_details()
16     interpreter.set_tensor(inp[0]["index"], arr)
17     interpreter.invoke()
18     preds = interpreter.get_tensor(out[0]["index"])[0]
19     i = int(np.argmax(preds))
20     return classes[i], float(preds[i]) * 100
```

Listing B.1 – Chargement et inférence du modèle TFLite

B.2 — Statistiques du module Historique

```
1  def statistiques():
2      conn = sqlite3.connect("historique.db")
3      n_irr = conn.execute(
```

```

4      "SELECT COUNT(*) FROM irrigation").fetchone()[0]
5      n_fert = conn.execute(
6      "SELECT COUNT(*) FROM fertilisation").fetchone()[0]
7      n_mal  = conn.execute(
8      "SELECT COUNT(*) FROM maladies").fetchone()[0]
9      top = conn.execute("""
10     SELECT culture, COUNT(*) AS n FROM (
11     SELECT culture FROM irrigation
12     UNION ALL SELECT culture FROM fertilisation
13     UNION ALL SELECT culture FROM maladies)
14     GROUP BY culture ORDER BY n DESC LIMIT 1
15     """).fetchone()
16     conn.close()
17     return {"total": n_irr + n_fert + n_mal,
18           "culture_top": top[0] if top else "-"}

```

Listing B.2 – Calcul des statistiques globales de l’historique

B.3 — Export CSV

```

1      import pandas as pd, streamlit as st
2
3      df  = pd.DataFrame(lire_historique_irrigation(1000))
4      csv = df.to_csv(index=False).encode("utf-8")
5      st.download_button(
6      label="Exporter en CSV",
7      data=csv,
8      file_name="historique_irrigation.csv",
9      mime="text/csv")

```

Listing B.3 – Export CSV d’une table de l’historique (Streamlit)

Annexe C

Diagrammes UML complets

Les diagrammes UML détaillés (cas d'utilisation avec relations *include*, diagrammes de séquence des quatre modules, diagramme de classes complet) sont disponibles dans le dépôt du projet [14] :

https://github.com/tourabiissakhissein33-dev/PFE_Agriculture_IA

Annexe D

Captures d'écran de l'application

Les figures ci-dessous présentent les principales interfaces de l'application.

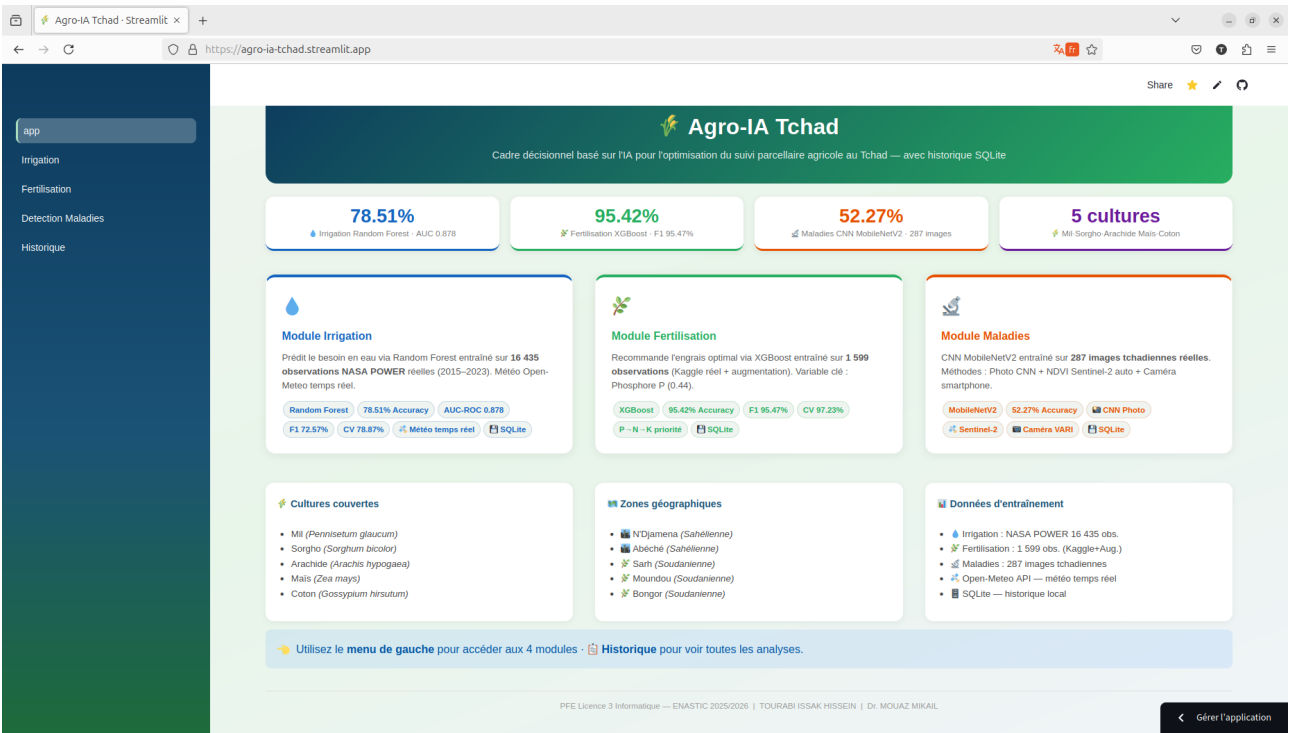


FIGURE D.1 : Page d'accueil de l'application Agro-IA Tchad

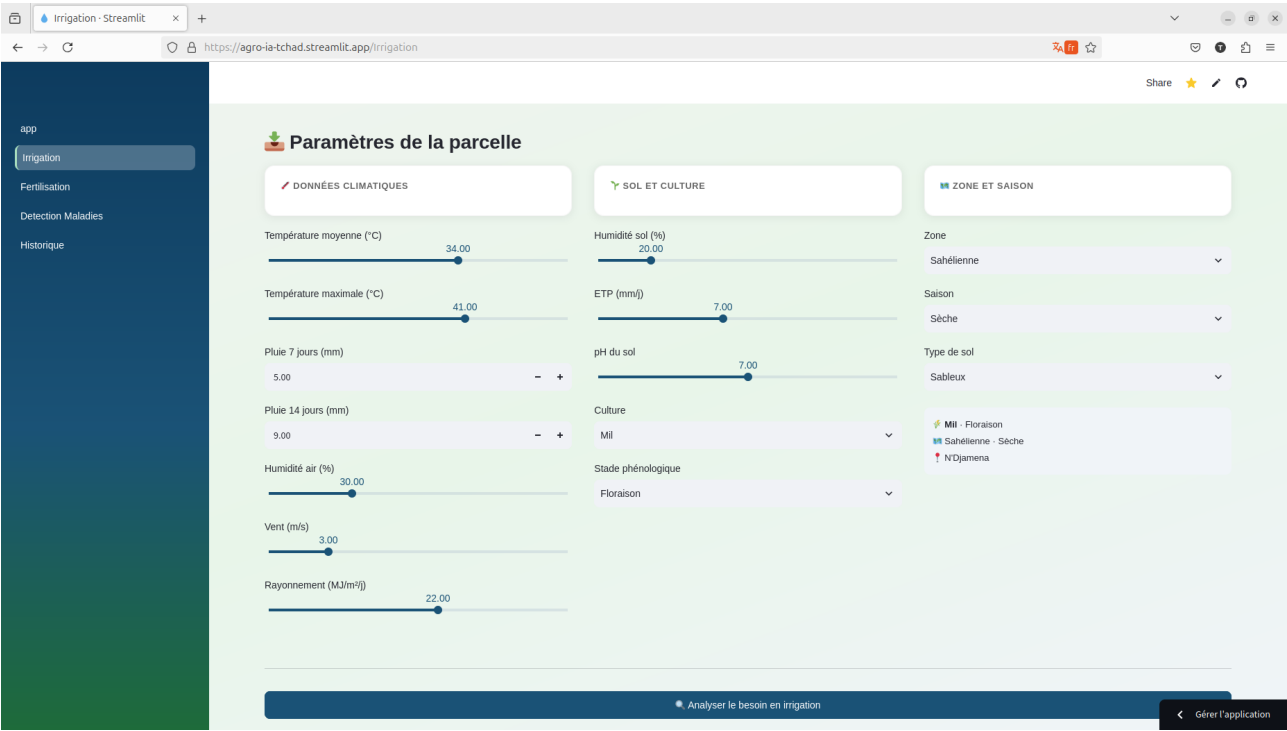


FIGURE D.2 : Module Irrigation — analyse et recommandation

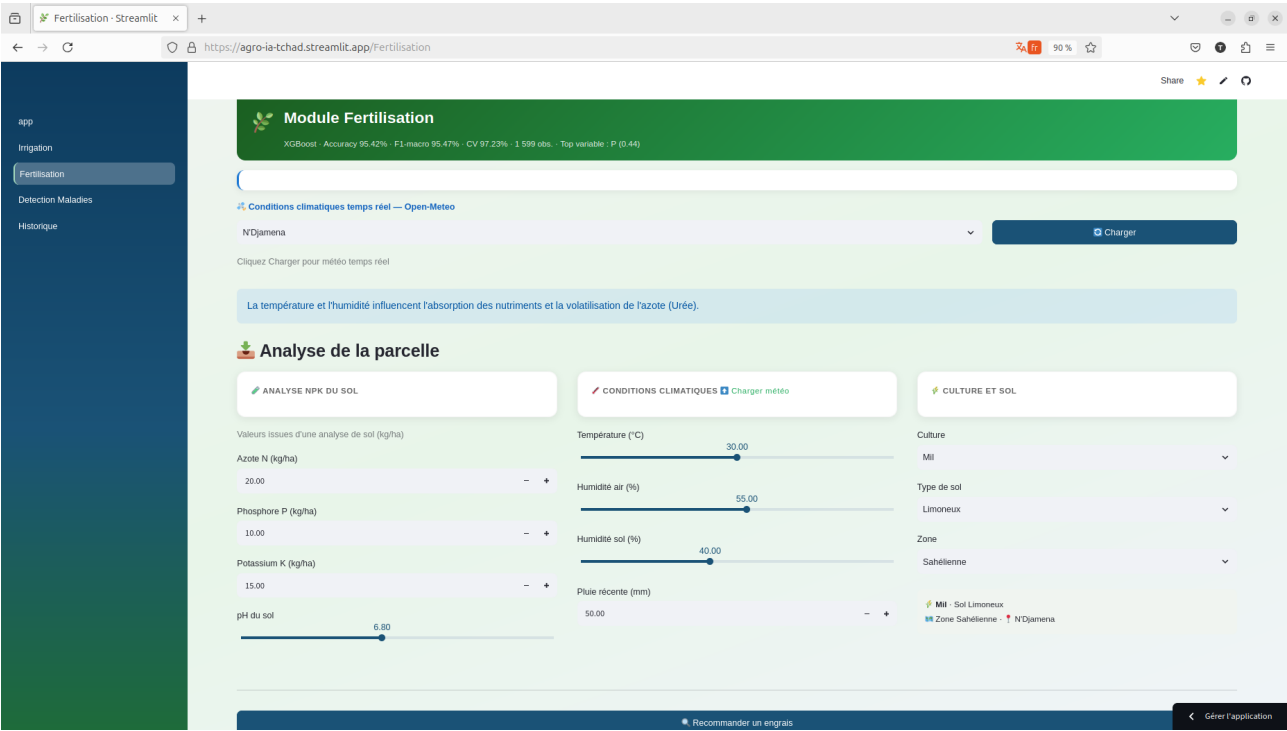


FIGURE D.3 : Module Fertilisation — recommandation d’engrais

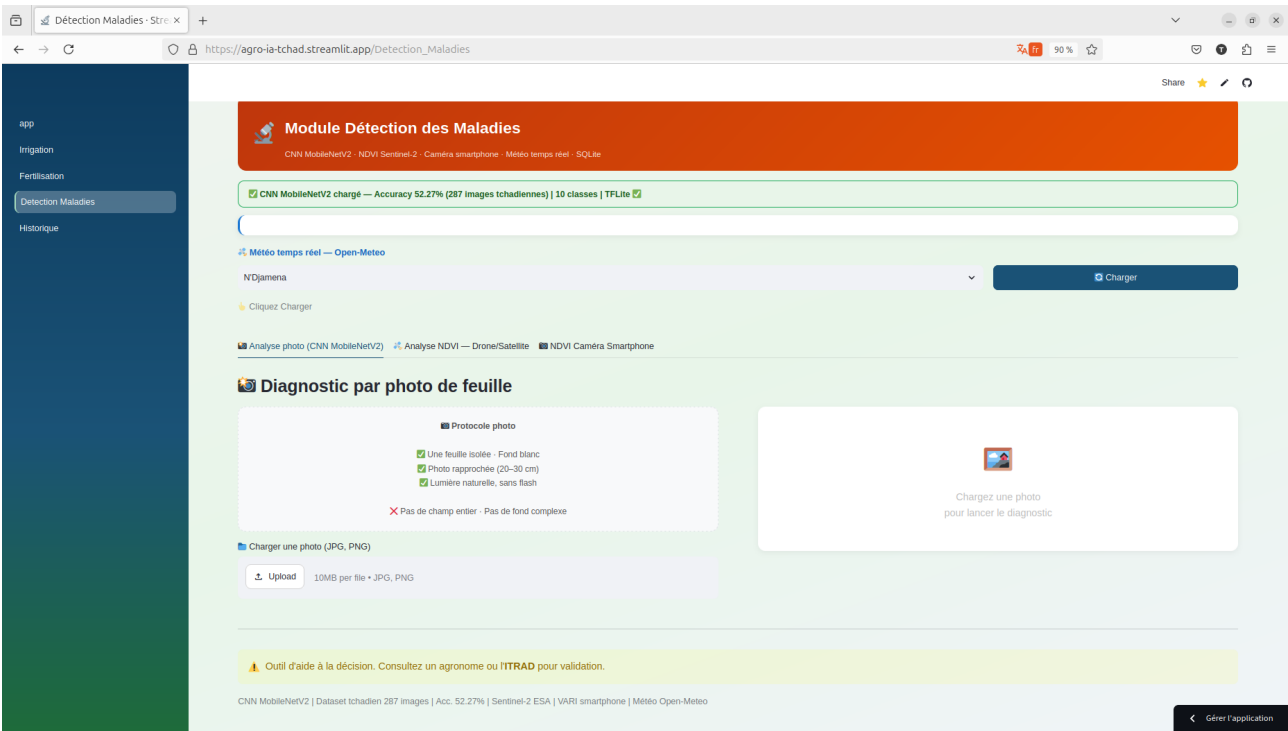


FIGURE D.4 : Module Détection des maladies — diagnostic par photo

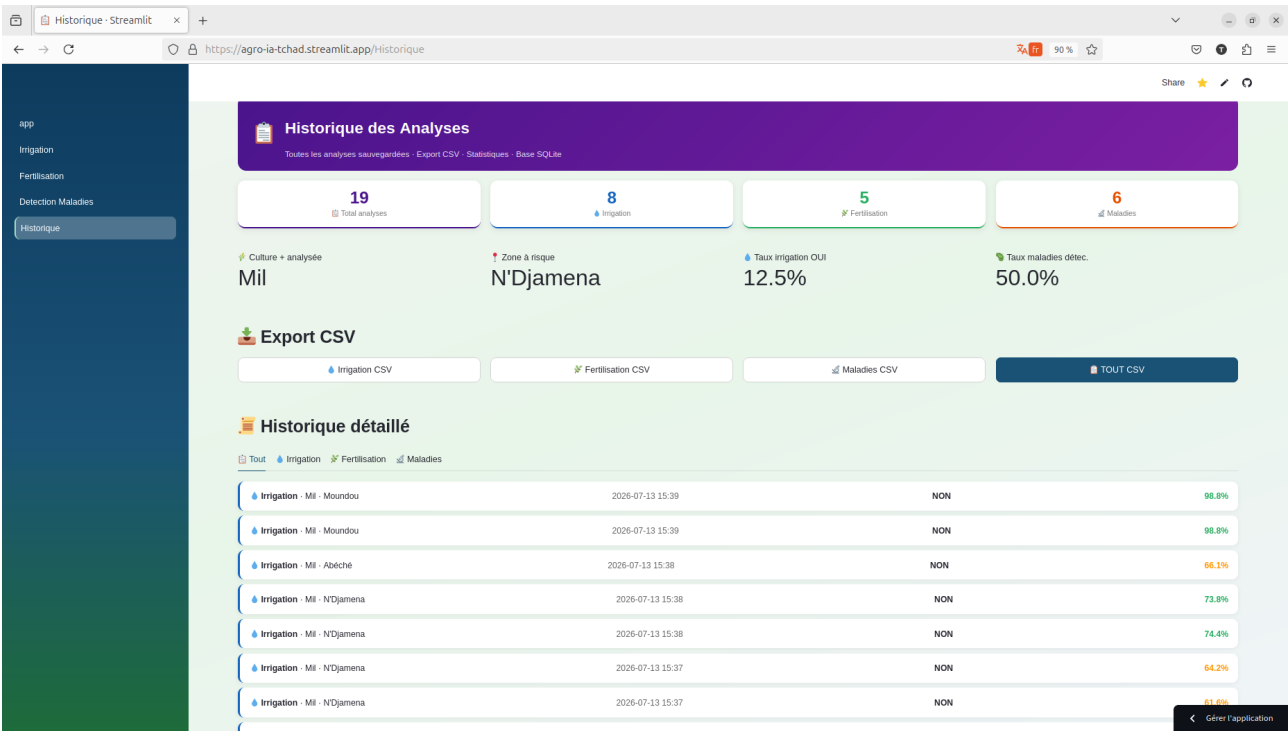


FIGURE D.5 : Module Historique — consultation et export

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Sommaire	v
Liste des figures	vii
Liste des tableaux	viii
Liste des abréviations	ix
Introduction Générale	1
1. Contexte général	1
2. Problématique	1
3. Objectifs du travail	2
4. Motivation et intérêt du sujet	2
5. Méthodologie adoptée	3
6. Structure du mémoire	3
1 Travaux existants	4
Introduction	4
1.1 Agriculture de précision et suivi parcellaire	4
1.1.1 Définition et principes	4
1.1.2 Le suivi parcellaire	4
1.1.3 Contexte agricole du Tchad	5
1.1.4 Défis du suivi parcellaire en zone sahélienne	5
1.2 Données agricoles : nature et hétérogénéité	5
1.2.1 Données climatiques	5
1.2.2 Données pédologiques	5
1.2.3 Données visuelles et télédétection	6

1.2.4	L'indice de végétation NDVI	6
1.3	Intelligence artificielle appliquée à l'agriculture	6
1.3.1	Apprentissage automatique classique	6
1.3.2	Apprentissage profond et vision par ordinateur	6
1.3.3	Transfert d'apprentissage et modèles légers	7
1.4	Analyse des solutions existantes et positionnement	7
1.4.1	Solutions existantes	7
1.4.2	Limites des solutions existantes	7
1.4.3	Positionnement de notre contribution	8
	Conclusion	8
2	Analyse, Conception et Modélisation UML	9
	Introduction	9
2.1	Analyse du problème et identification des acteurs	9
2.1.1	Analyse du problème	9
2.1.2	Identification des acteurs	9
2.2	Spécification des besoins	10
2.2.1	Besoins fonctionnels	10
2.2.2	Besoins non fonctionnels	10
2.3	Modélisation UML du système	11
2.3.1	Diagramme de contexte	11
2.3.2	Diagramme de cas d'utilisation	12
2.3.3	Diagramme de séquence — analyse d'une parcelle	13
2.3.4	Diagramme d'activités — génération d'une recommandation	13
2.4	Architecture du cadre décisionnel	14
2.5	Modèle de la base de données de l'historique	16
	Conclusion	17
3	Implémentation et Tests	18
	Introduction	18
3.1	Environnement de développement	18
3.2	Collecte et prétraitement des données	19
3.2.1	Données d'irrigation — NASA POWER	19
3.2.2	Données de fertilisation	19
3.2.3	Dataset local d'images — maladies	20
3.3	Module IA — Irrigation	20
3.3.1	Choix de l'algorithme Random Forest	20
3.3.2	Entraînement et hyperparamètres	20
3.3.3	Intégration dans l'application	20
3.4	Module IA — Fertilisation	21
3.4.1	Choix de l'algorithme XGBoost	21

3.4.2	Entrées et sorties du module	21
3.5	Module IA — Détection des maladies	21
3.5.1	Architecture CNN MobileNetV2 et transfert d'apprentissage	21
3.5.2	Conversion au format TensorFlow Lite	21
3.5.3	Analyse NDVI : satellite et caméra smartphone	22
3.6	Module Historique — Persistance SQLite et export CSV	22
3.6.1	Implémentation de la couche d'accès aux données	22
3.6.2	Consultation, statistiques et export	23
3.7	Développement de l'application web	23
3.7.1	Framework Streamlit	23
3.7.2	Intégration de la météo temps réel	23
3.7.3	Interface et expérience utilisateur	23
3.8	Protocole de tests	24
3.8.1	Métriques d'évaluation des modèles	24
3.8.2	Démarche de validation des modèles	24
3.8.3	Plan de tests fonctionnels de l'application	24
	Conclusion	24
4	Résultats et Discussion	25
	Introduction	25
4.1	Résultats du module Irrigation	25
4.2	Résultats du module Fertilisation	26
4.3	Résultats du module Détection des maladies	26
4.4	Résultats des tests du module Historique	27
4.5	Discussion générale des résultats	27
4.6	Déploiement de l'application	28
4.6.1	Déploiement local	28
4.6.2	Déploiement sur Streamlit Cloud	28
	Conclusion	28
	Conclusion Générale et Perspectives	29
	Synthèse des travaux	29
	Limites rencontrées	29
	Perspectives d'amélioration	30
	Bibliographie	31
A	Jeux de données utilisés	33
B	Codes sources principaux	35
C	Diagrammes UML complets	37

D Captures d'écran de l'application	38
Table des matières	41